

Lecture 3: R Basics Continued

Heather Mattie, PhD

September 8, 2026

Recipe of the Day!

Lemon blueberry scones



Office Hours and Lab

Office Hours

Day	Time	Staff	Location
Thursday	12-1 pm	Heather	Building 1, 4 th floor, room 421A
Wednesday	10:30-11:30 am	Stephanie	Building 2, 4 th floor, room 426
Friday	11:30-12:30 pm	Monica	Building 2, 4 th floor, room 426
Monday	1-2 pm	Kimberly	Zoom

Lab

Day	Time	Location
Friday	9:45-11:15 am	Zoom

***Lab will not be held every week**



Announcements

- Lab **will** be held this week
- Homework #1 due Friday, September 18th by 11:59pm
 - May change depending on pacing
 - Can use up to 2 late days

Homework #1 details

- Your plots can be created using base R. You do not need to use `ggplot2` - yet. You will for all other assignments.
- You do not need to use the pipe (`%>%`) for this homework. You will for all other assignments.
- Comment your code, i.e., provide short explanations of what each chunk of code is doing. This is good coding practice. Everyone who reads your code – including your future self – will thank you.

General homework tips

- **Do not** write text answers inside of a code chunk. Code chunks are for code (and code comments) only. This is the main reason we are using R Markdown files!
- Be sure your homework knits successfully and that you commit and push **BOTH** the .Rmd and .html files.
- **Do not** print out an entire dataset. Use the `head()` or `glimpse()` functions.

R Basics



R Basics Roadmap

1. Data types
2. Vectors
3. Sorting
4. Indexing
5. Basic plots
6. Programming basics

R Basics

Vectors

Heather Mattie, PhD

Creating vectors

Vectors are created using the concatenate function **c**.

Vectors with text or characters as entries must have quotation marks around each entry.

- Without quotation marks, R will look for variables with those names
- An error will be produced

```
codes <- c(380, 124, 818)
codes
```

```
## [1] 380 124 818
```

```
country <- c("italy", "canada", "egypt")
country
```

```
## [1] "italy" "canada" "egypt"
```

```
country <- c(italy, canada, egypt)
```

Names

It might be useful to **name** the entries of a vector

- Connect the entry with a name for the entry with an equal sign
- Names can be coded with or without quotation marks
- Names can also be assigned using the **names** function

```
codes <- c(italy = 380, canada = 124, egypt = 818)
codes
```

```
## italy canada egypt
## 380 124 818
```

```
names(codes)
```

```
## [1] "italy" "canada" "egypt"
```

```
codes <- c("italy" = 380, "canada" = 124, "egypt" = 818)
codes
```

```
## italy canada egypt
## 380 124 818
```

```
codes <- c(380, 124, 818)
country <- c("italy", "canada", "egypt")
names(codes) <- country
codes
```

```
## italy canada egypt
## 380 124 818
```

Sequences

The **seq** function creates vector sequences

- The first argument defines the start
- The second argument defines the end
- The default increment is to increase by 1 but this can be customized
- There is shorthand for consecutive numbers
- The default class of the entries in a sequence is integer but will be numeric if decimal numbers are part of the sequence

```
seq(1, 10)
```

```
## [1] 1 2 3 4 5 6 7 8 9 10
```

```
seq(1, 10, 2)
```

```
## [1] 1 3 5 7 9
```

```
1:10
```

```
## [1] 1 2 3 4 5 6 7 8 9 10
```

```
class(1:10)
```

```
## [1] "integer"
```

```
class(seq(1, 10, 0.5))
```

```
## [1] "numeric"
```

Subsetting

Square brackets are used to access specific elements in a vector

- The position of the entry in the vector, or the name of the entry can be used to access it
- A vector or sequence of numbers can also be used inside the square brackets

```
codes
```

```
## italy canada egypt  
## 380 124 818
```

```
codes[2]
```

```
## canada  
## 124
```

```
codes[c(1,3)]
```

```
## italy egypt  
## 380 818
```

```
codes[1:2]
```

```
## italy canada  
## 380 124
```

```
codes["canada"]
```

```
## canada  
## 124
```

```
codes[c("egypt", "italy")]
```

```
## egypt italy  
## 818 380
```

Coercion

Coercion is R attempting to be flexible with data types

- The entries in a vector must be the same data type
- If an entry isn't the type expected, R will guess what it is and **coerce** the value to be a certain type if it can
- Otherwise, it will give an error
- Numeric entries **can** be coerced into characters
- Character entries **cannot** be coerced into numbers

```
x <- c(1, "canada", 3)
x
```

```
## [1] "1"      "canada" "3"
```

```
class(x)
```

```
## [1] "character"
```

```
x <- 1:5
y <- as.character(x)
y
```

```
## [1] "1" "2" "3" "4" "5"
```

```
as.numeric(y)
```

```
## [1] 1 2 3 4 5
```

Not Availables (NA)

When a coercion function encounters an impossible case, it gives a warning and turns the entry into an **NA**

- NA stands for not available
- NA is also used to denote missing data

```
x <- c("1", "b", "3")  
as.numeric(x)
```

```
## Warning: NAs introduced by coercion
```

```
## [1] 1 NA 3
```

Vector arithmetic

Vector arithmetic involves performing mathematical operations on vectors

- Addition, subtraction, multiplication, division, etc.
- In R, arithmetic operations on vectors occur *element wise*

```
heights <- c(69, 62, 66, 70, 70, 73, 67, 73, 67, 70)
```

```
heights * 2.54
```

```
## [1] 175.26 157.48 167.64 177.80 177.80 185.42 170.18 185.42 170.18 177.80
```

```
heights - 69
```

```
## [1] 0 -7 -3 1 1 4 -2 4 -2 1
```

Two vectors

If we have two vectors of the same length, and we sum them, they are added entry by entry

- The same holds for subtraction, multiplication, and division

$$\begin{pmatrix} a \\ b \\ c \\ d \end{pmatrix} + \begin{pmatrix} e \\ f \\ g \\ h \end{pmatrix} = \begin{pmatrix} a + e \\ b + f \\ c + g \\ d + h \end{pmatrix}$$

Summary

- Vectors are objects with multiple entries that can be created with the concatenate function
- All entries in a vector must be of the same data type
- The entries in a vector can be named
- The **seq** function produces a sequence vector
- Square brackets are used to subset a vector
- R will use coercion if the data types of entries in a vector conflict

R Basics

Sorting

Heather Mattie, PhD

sort

The **sort** function sorts a vector in **increasing** order

- It does not indicate the placement, or index, of the values once they have been sorted

```
library(dslabs)
data(murders)
sort(murders$total)
```

```
## [1] 2 4 5 5 7 8 11 12 12 16 19 21 22 27 32
## [16] 36 38 53 63 65 67 84 93 93 97 97 99 111 116 118
## [31] 120 135 142 207 219 232 246 250 286 293 310 321 351 364 376
## [46] 413 457 517 669 805 1257
```

order

The **order** function takes a vector and returns a vector of indices

- The indices show how to sort the original vector
- It does not sort the data itself

```
x <- c(31, 4, 15, 92, 65)
sort(x)
```

```
## [1] 4 15 31 65 92
```

```
index <- order(x)
x[index]
```

```
## [1] 4 15 31 65 92
```

```
x
```

```
## [1] 31 4 15 92 65
```

```
order(x)
```

```
## [1] 2 3 1 5 4
```

order

```
library(dslabs)
data(murders)
ind <- order(murders$total)
murders$abb[ind]
```

```
## [1] "VT" "ND" "NH" "WY" "HI" "SD" "ME" "ID" "MT" "RI" "AK" "IA" "UT" "WV" "NE"
## [16] "OR" "DE" "MN" "KS" "CO" "NM" "NV" "AR" "WA" "CT" "WI" "DC" "OK" "KY" "MA"
## [31] "MS" "AL" "IN" "SC" "TN" "AZ" "NJ" "VA" "NC" "MD" "OH" "MO" "LA" "IL" "GA"
## [46] "MI" "PA" "NY" "FL" "TX" "CA"
```

Rate example using **order**

Vector arithmetic can be especially useful for creating new variables quickly

- Example: creating and comparing murder rates for each state rather than the total number of murders in a particular state

```
head(murders)
```

```
##      state abb region population total
## 1  Alabama AL  South   4779736    135
## 2  Alaska  AK   West    710231     19
## 3  Arizona AZ   West   6392017    232
## 4  Arkansas AR  South   2915918     93
## 5 California CA   West  37253956   1257
## 6  Colorado CO   West   5029196     65
```

```
murder_rate <- murders$total / murders$population * 100000
```

```
murders$state[order(murder_rate)]
```

```
## [1] "Vermont"           "New Hampshire"    "Hawaii"
## [4] "North Dakota"      "Iowa"             "Idaho"
## [7] "Utah"              "Maine"            "Wyoming"
## [10] "Oregon"            "South Dakota"     "Minnesota"
## [13] "Montana"           "Colorado"         "Washington"
## [16] "West Virginia"     "Rhode Island"     "Wisconsin"
## [19] "Nebraska"          "Massachusetts"    "Indiana"
## [22] "Kansas"            "New York"         "Kentucky"
## [25] "Alaska"            "Ohio"             "Connecticut"
## [28] "New Jersey"        "Alabama"          "Illinois"
## [31] "Oklahoma"          "North Carolina"   "Nevada"
## [34] "Virginia"          "Arkansas"         "Texas"
## [37] "New Mexico"        "California"       "Florida"
## [40] "Tennessee"         "Pennsylvania"     "Arizona"
## [43] "Georgia"           "Mississippi"      "Michigan"
## [46] "Delaware"          "South Carolina"   "Maryland"
## [49] "Missouri"          "Louisiana"        "District of Columbia"
```

rank

For any given vector, the **rank** function gives you a vector with the rank of the first entry, second entry, etc. of the vector

```
x <- c(31, 4, 15, 92, 65)  
rank(x)
```

```
## [1] 3 1 2 5 4
```

Comparing sorting functions

original	sort	order	rank
31	4	2	3
4	15	3	1
15	31	1	2
92	65	5	5
65	92	4	4

Summary

- The **sort** function sorts a vector in increasing order
- The **order** function returns a vector of indices that sort the original vector
- The **rank** function returns a vector with the rank of each entry when the original vector is sorted

R Basics

Indexing

Heather Mattie, PhD

murders dataset

```
head(murders)
```

```
##      state abb region population total
## 1  Alabama  AL  South    4779736    135
## 2   Alaska  AK   West     710231     19
## 3  Arizona  AZ   West    6392017    232
## 4 Arkansas  AR   South    2915918     93
## 5 California CA   West   37253956   1257
## 6  Colorado CO   West    5029196     65
```

max and which.max

The **max** function returns the maximum value in a vector

The **which.max** function returns the index of the maximum value

```
max(murders$total)
```

```
## [1] 1257
```

```
i_max <- which.max(murders$total)  
i_max
```

```
## [1] 5
```

```
murders$state[i_max]
```

```
## [1] "California"
```

Subsetting with logicals

Logical statements (**logicals**) can be used to index vectors

- Return **TRUE** or **FALSE** for each entry in a vector
- **TRUE** values are coded as **1**
- **FALSE** values are coded as **0**

```
murder_rate <- murders$total / murders$population * 100000  
ind <- murder_rate < 0.71  
ind
```

```
## [1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE  
## [13] FALSE FALSE FALSE TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE  
## [25] FALSE FALSE FALSE FALSE FALSE TRUE FALSE FALSE FALSE FALSE TRUE FALSE  
## [37] FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE FALSE FALSE  
## [49] FALSE FALSE FALSE
```

```
sum(ind)
```

```
## [1] 5
```

```
murders$state[ind]
```

```
## [1] "Hawaii"      "Iowa"         "New Hampshire" "North Dakota"  
## [5] "Vermont"
```

Logical operators

Logical operators are symbols or words used to connect two or more expressions

- Result in a **TRUE** or **FALSE**
- AND = **&**
- OR = **|**
- NOT = **!**

```
TRUE & TRUE
```

```
## [1] TRUE
```

```
TRUE & FALSE
```

```
## [1] FALSE
```

```
FALSE & FALSE
```

```
## [1] FALSE
```

```
west <- murders$region == "West"  
safe <- murder_rate <= 1  
ind <- safe & west  
murders$state[ind]
```

```
## [1] "Hawaii" "Idaho" "Oregon" "Utah" "Wyoming"
```

Logical operators

Logical operators are symbols or words used to connect two or more expressions

- Result in a **TRUE** or **FALSE**
- AND = **&**
- OR = **|**
- NOT = **!**

```
ind <- safe | west  
murders$state[ind]
```

```
## [1] "Alaska"      "Arizona"      "California"    "Colorado"  
## [5] "Hawaii"      "Idaho"        "Iowa"          "Maine"  
## [9] "Minnesota"   "Montana"      "Nevada"        "New Hampshire"  
## [13] "New Mexico"  "North Dakota" "Oregon"        "South Dakota"  
## [17] "Utah"        "Vermont"      "Washington"    "Wyoming"
```

```
ind <- safe & !west  
murders$state[ind]
```

```
## [1] "Iowa"        "Maine"        "Minnesota"     "New Hampshire"  
## [5] "North Dakota" "South Dakota" "Vermont"
```

which

The **which** function indicates which entries of a logical vector are **TRUE**

```
ind <- which(murders$state == "California")  
ind # this is the index that matches the California entry
```

```
## [1] 5
```

```
murder_rate[ind]
```

```
## [1] 3.374138
```

match

The **match** function indicates which indices of a second vector match each of the entries of a first vector

```
ind <- match(c("New York", "Florida", "Texas"), murders$state)
ind
```

```
## [1] 33 10 44
```

```
match(c("Boston", "Dakota", "Washington"), murders$state)
```

```
## [1] NA NA 48
```

```
murder_rate[ind]
```

`%in%`

The `%in%` function indicates whether or not each element in a vector is contained in another vector

```
c("Boston", "Dakota", "Washington") %in% murders$state
```

```
## [1] FALSE FALSE  TRUE
```

Summary

- Vectors can be subset with logical statements
- Logical operators (**&**, **|**, **!**) can connect two or more logical statements
- The **which**, **match** and **%in%** functions can be used to index where TRUE values are in a vector